

Indexing

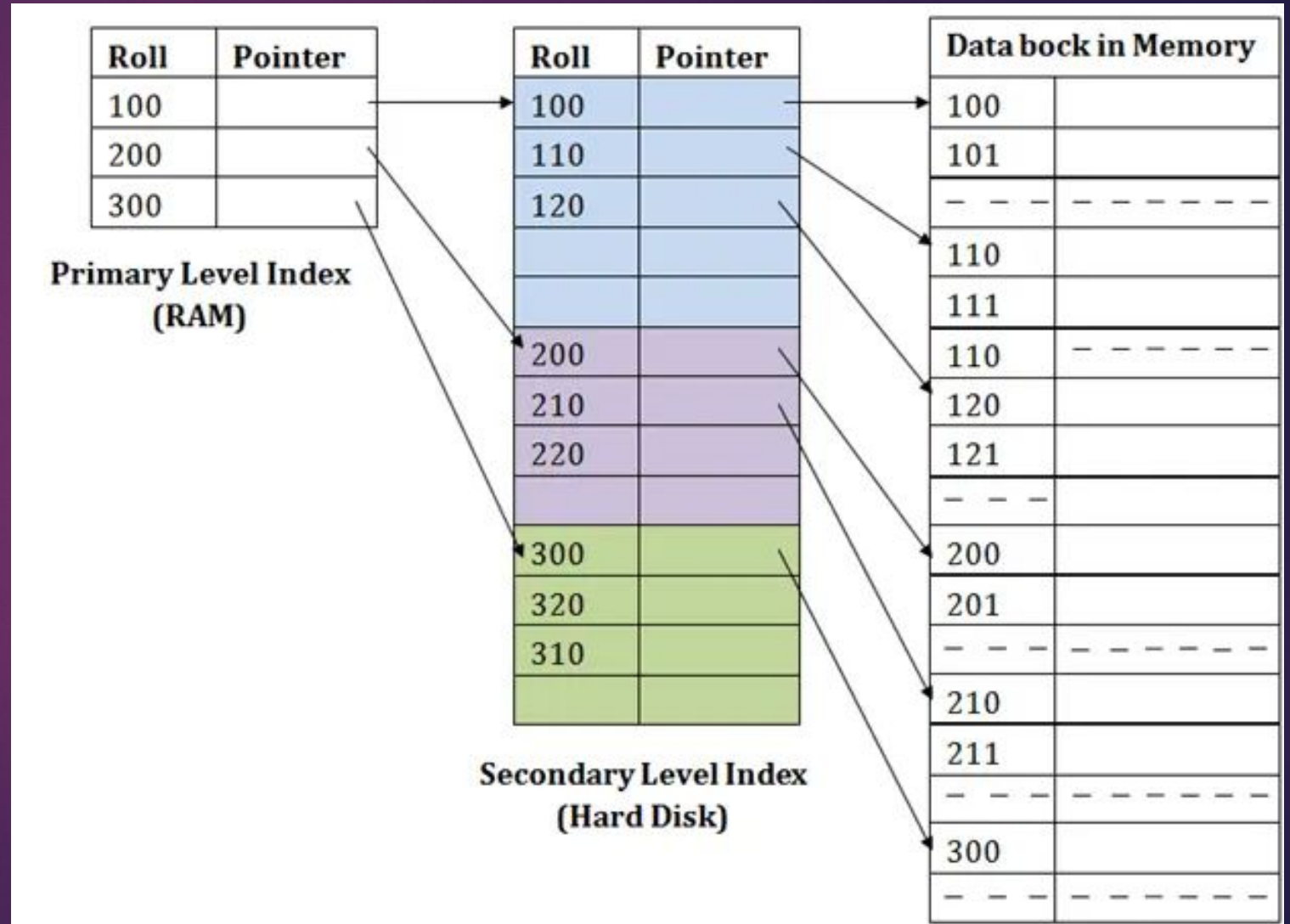
K KRISHNA PRIYA

Indexing in DBMS?

- ▶ In the world of databases, efficient data retrieval is crucial for optimal performance and user satisfaction.
- ▶ Indexing plays a significant role in improving query execution speed by enabling rapid access to data.
- ▶ Indexing refers to a data structure technique that is used for quickly retrieving entries from database files using some attributes that have been indexed.
- ▶ In database systems, indexing is comparable to indexing in books. The indexing attributes are used to define the indexing.

What is Indexing

- ▶ In simple terms, indexing in DBMS is the process of creating a data structure, known as an index, which allows for quick access to specific data records within a database table.
- ▶ Just like an index in a book helps you find relevant information quickly, database indexes enhance search and retrieval operations.



How Does Indexing Work?

- ▶ let's consider an example. Suppose we have a database table called "Employees" with thousands of records.
- ▶ Without an index, the DBMS would have to scan through every record in the table to find specific information, resulting in slower query execution.
- ▶ With indexing, however, the DBMS creates an additional structure, known as an index, which holds a sorted copy of selected columns from the table.
- ▶ This index structure includes key-value pairs, where the key represents the data column(s) being indexed, and the value is the corresponding pointer to the actual record location.
- ▶ By using this index structure, the DBMS can directly locate the desired data, eliminating the need for a full table scan.

Benefits of Indexing:

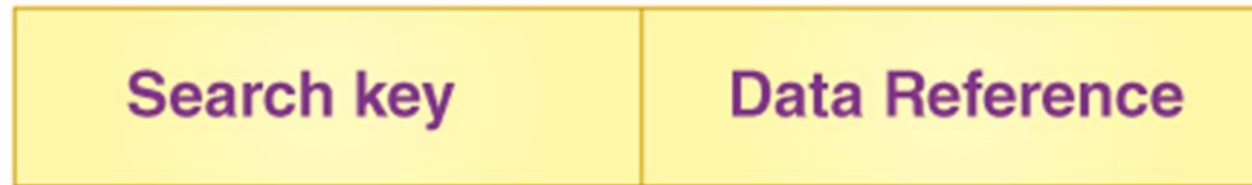
- ▶ **Improved Query Performance:** Indexing accelerates query execution by reducing the time required to locate specific data within a table, resulting in faster response times.
- ▶ **Efficient Data Retrieval:** With indexes, the DBMS can quickly narrow down the search space, leading to more efficient data retrieval operations.
- ▶ **Enhanced Data Integrity:** Indexes can enforce uniqueness and primary key constraints, ensuring data integrity and preventing duplicate entries.
- ▶ **Optimized Sorting and Grouping:** Indexes facilitate efficient sorting and grouping operations, enabling faster processing of ordered and aggregated data.
- ▶ **Flexibility in Data Access:** Indexes allow for different access paths to the data, enabling the DBMS to choose the most appropriate one based on the query.

Structure of Index

- ▶ We can create indices using some columns of the database.
- ▶ The search key is the database's first column, and it contains a duplicate or copy of the table's candidate key or primary key. The primary key values are saved in sorted order so that the related data can be quickly accessible.
- ▶ The data reference is the database's second column. It contains a group of pointers that point to the disk block where the value of a specific key can be found.

B-tree -Indexing

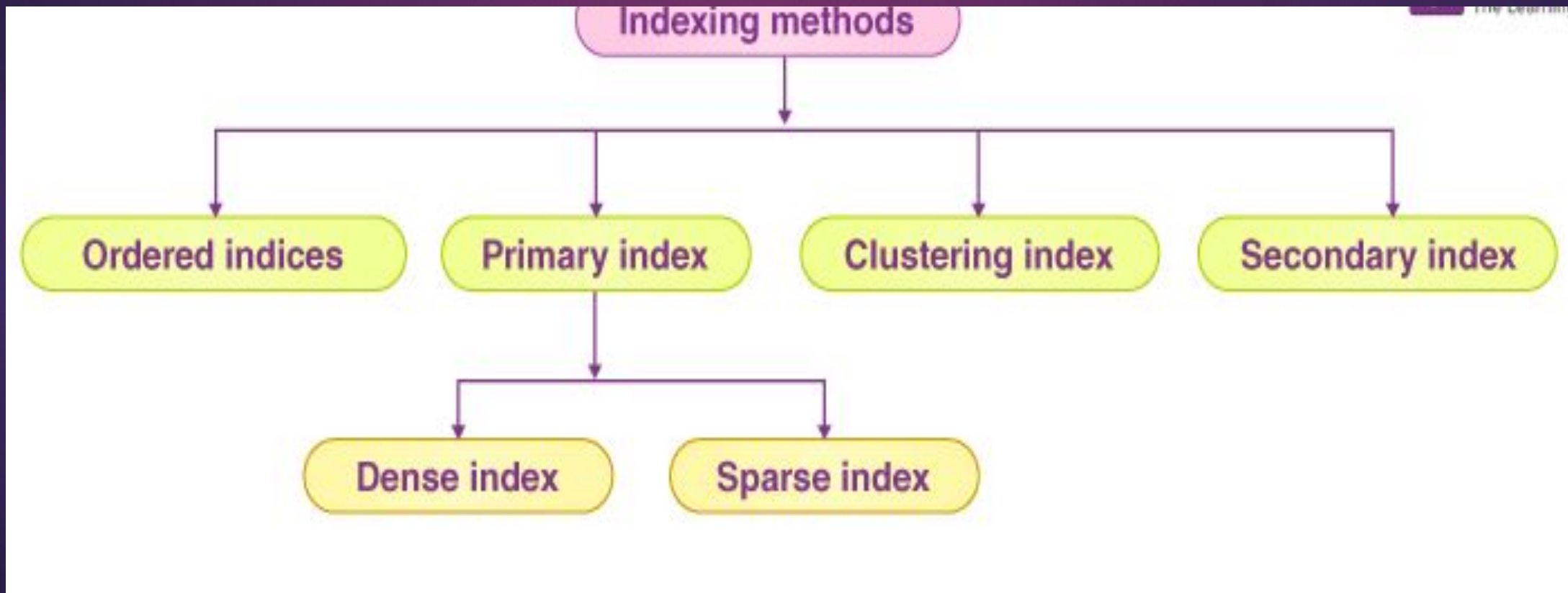
- ▶ B-tree-indexing is one of the most popular and commonly used indexing techniques. B-tree in DBMS is a type of tree data structure that contains 2 things namely: **Index Key** and its corresponding **disk address**. Index Key refers to a certain disk address and that disk further contains rows or tuples of data.



Structure of Index

Methods of Indexing

- ▶ In general, there are different types of file organization mechanisms that are followed by the indexing methods



Single Level Indexing

- ▶ It is somewhat like the index (or the table of contents) found in a book. Index of a book contains topic names along with the page number similarly the index table of the database contains keys and their corresponding block address.

field 1	field 2
5	
4	

Primary index

5	
6	
7	
3	
4	
23	
27	
55	

Data File

Block 1

Block 2

Sequential File Organization or Ordered Index File

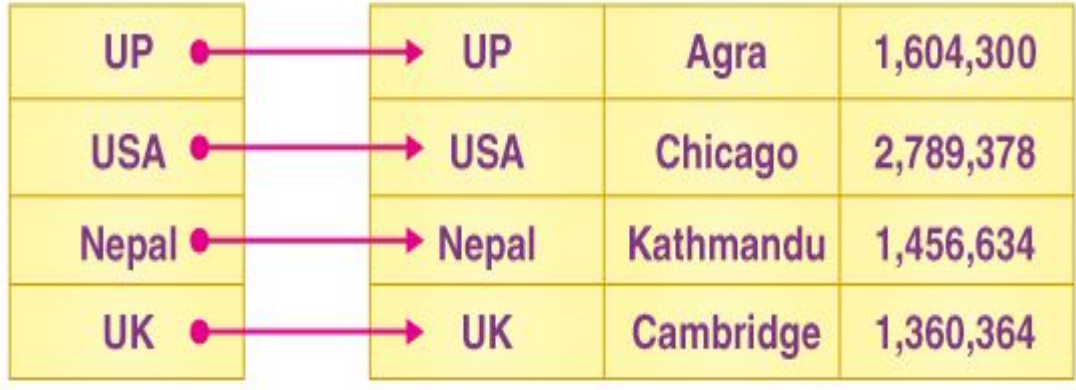
- ▶ To make searching easier and faster, the indices are frequently arranged/sorted. Ordered indices are indices that have been sorted.
- ▶ **Example**
- ▶ Let's say we have a table of employees with thousands of records, each of which is ten bytes large. If their IDs begin with 1, 2, 3,..., etc., and we are looking for the student with ID-543:
- ▶ We must search the disk block from the beginning till it reaches 543 in the case of a DB without an index. After reading $543 \times 10 = 5430$ bytes, the DBMS will read the record.
- ▶ We will perform the search using indices in the case of an index, and the DBMS would read the record after it reads $542 \times 2 = 1084$ bytes, which is significantly less than the prior example.

Primary Index

- ▶ If the index is created on the basis of the primary key of the table, then it is known as primary indexing. These primary keys are unique to each record and contain 1:1 relation between the records.
- ▶ As primary keys are stored in sorted order, the performance of the searching operation is quite efficient.
- ▶ The primary index can be classified into two types: **Dense index** and **Sparse index**.

Dense index

- ▶ The dense index contains an index record for every search key value in the data file. It makes searching faster. In this, the number of records in the index table is same as the number of records in the main table. It needs more Space to store index record itself.



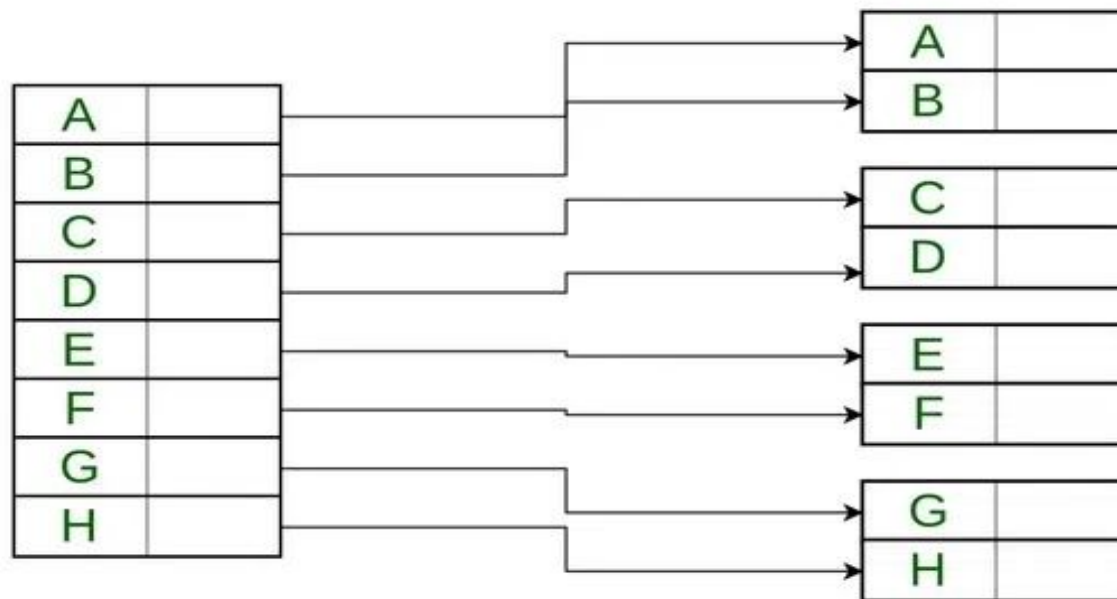
The diagram illustrates a dense index structure. On the left, a pointer table contains search keys (UP, USA, Nepal, UK) with red dots. Red arrows point from these dots to a corresponding index table. The index table has four columns: the search key, a pointer to the data key, the data key itself, and the data value. The data values are 1,604,300 for Agra, 2,789,378 for Chicago, 1,456,634 for Kathmandu, and 1,360,364 for Cambridge.

UP	→	UP	Agra	1,604,300
USA	→	USA	Chicago	2,789,378
Nepal	→	Nepal	Kathmandu	1,456,634
UK	→	UK	Cambridge	1,360,364

The index records have the search key and a pointer to the actual record on the disk.

Dence index Example 2

Dense Index



Data File

Index Record

For every search value in a Data File,

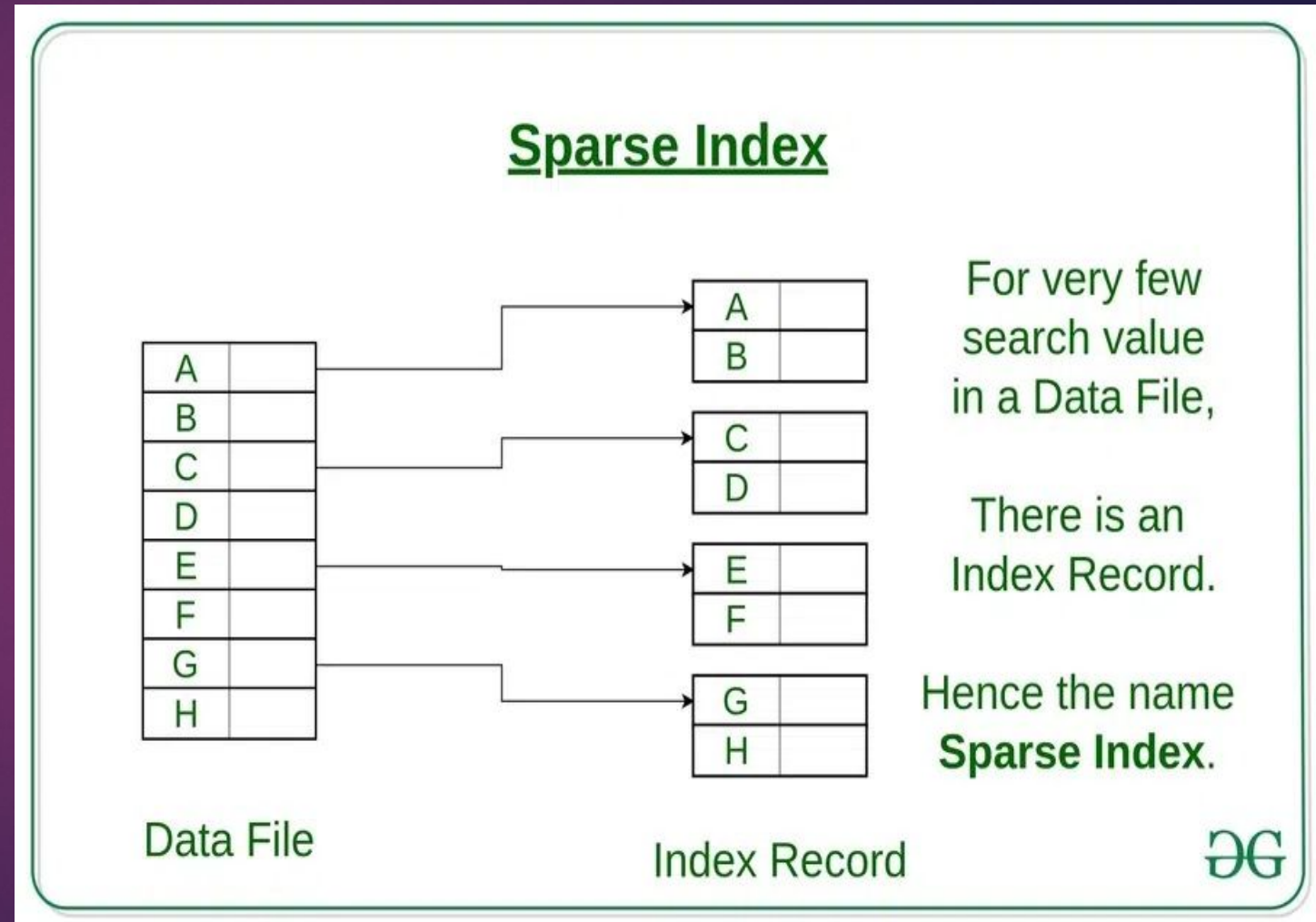
There is an Index Record.

Hence the name **Dense Index**.



Sparse Index

- ▶ The index record appears only for a few items in the data file. Each item points to a block as shown.
- ▶ To locate a record, we find the index record with the largest search key value less than or equal to the search key value we are looking for.
- ▶ We start at that record pointed to by the index record, and proceed along with the pointers in the file (that is, sequentially) until we find the desired record.
- ▶ Number of Accesses required = $\log_2(n) + 1$, (here n = number of blocks acquired by index file)



Clustering Index

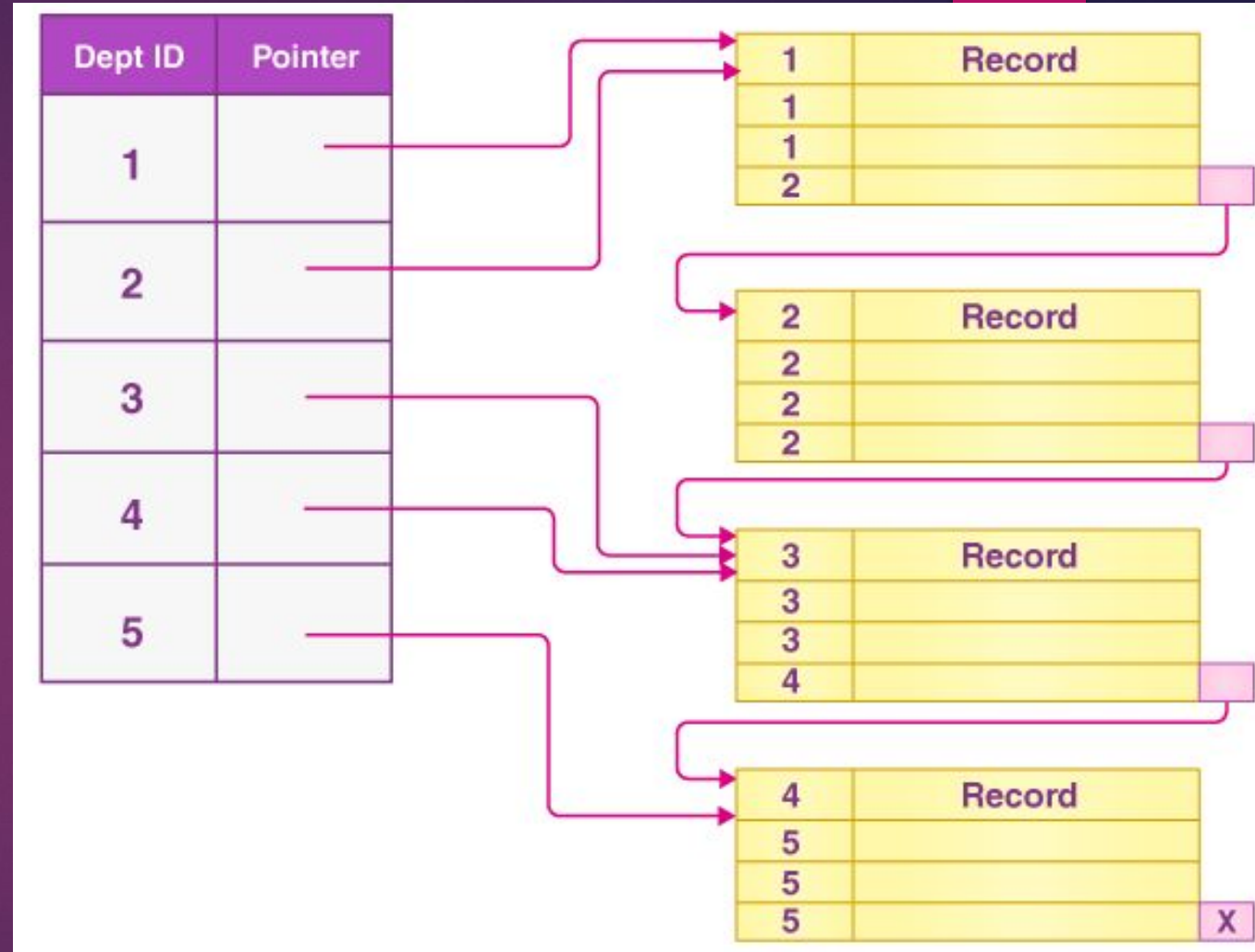
- ▶ An ordered data file can be defined as a clustered index. Non-primary key columns, which may or may not be unique for each record, are sometimes used to build indices.
- ▶ In this situation, we'll join two or more columns to acquire the unique value and generate an index out of them to make it easier to find the record. A clustering index is a name for this method.
- ▶ Records with comparable properties are grouped together, and indices for these groups are constructed.

Clustering Index

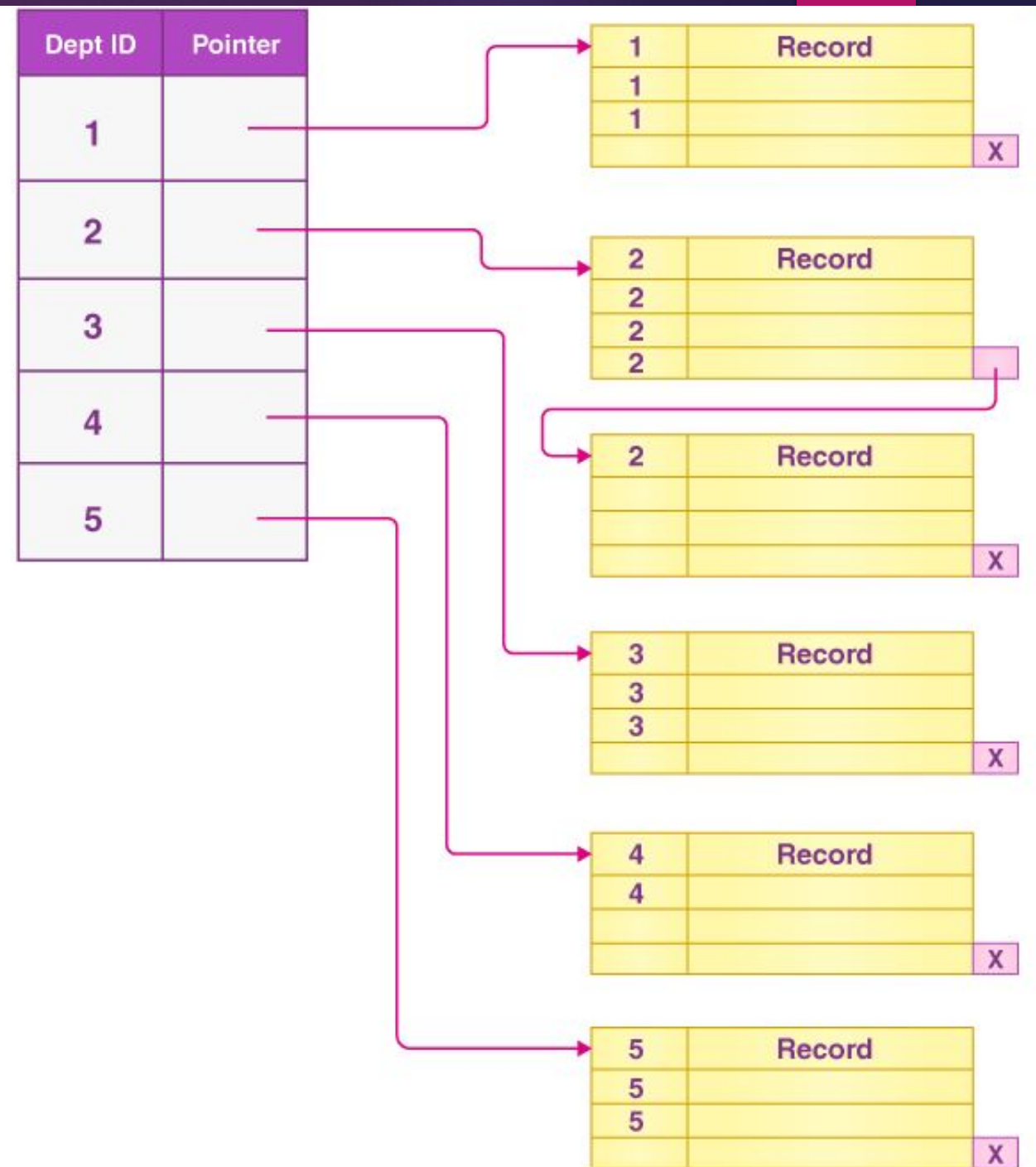
Assume that each department in a corporation has numerous employees.

Assume we utilise a clustering index, in which

all employees with the same Dept_ID are grouped together into a single cluster, and index pointers refer to the cluster as a whole. Dept_Id is a non-unique key in this case.



- ▶ Because one disk block is shared by records from various clusters, the previous structure is a little unclear.
- ▶ It is referred to as a better strategy when we employ distinct disk blocks for separate clusters.

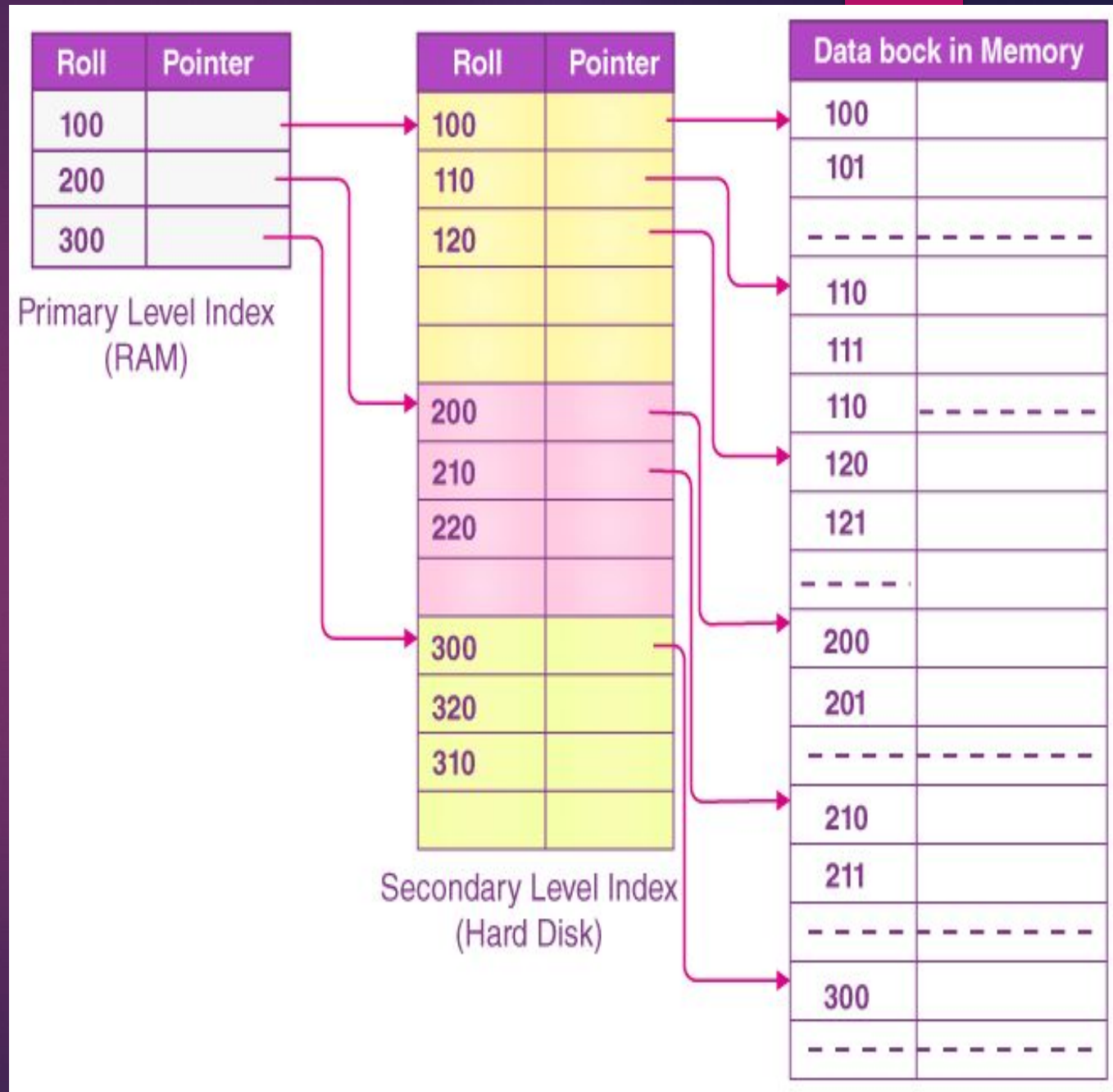


Secondary Index

- ▶ When using sparse indexing, the size of the mapping grows in sync with the size of the table. These mappings are frequently stored in primary memory to speed up address fetching.
- ▶ The secondary memory then searches the actual data using the address obtained through mapping.
- ▶ Fetching the address becomes slower as the mapping size increases. The sparse index will be ineffective in this scenario, so secondary indexing is used to solve this problem.
- ▶ Another level of indexing is introduced in secondary indexing to reduce the size of the mapping. The massive range for the columns is chosen first in this method, resulting in a small mapping size at the first level.
- ▶ Each range is then subdivided into smaller groups. Because the first level's mapping is kept in primary memory, fetching the addresses is faster.
- ▶ The second-level mapping, as well as the actual data, are kept in secondary memory (or hard disk) It is also called **Multi-Level Indexing**

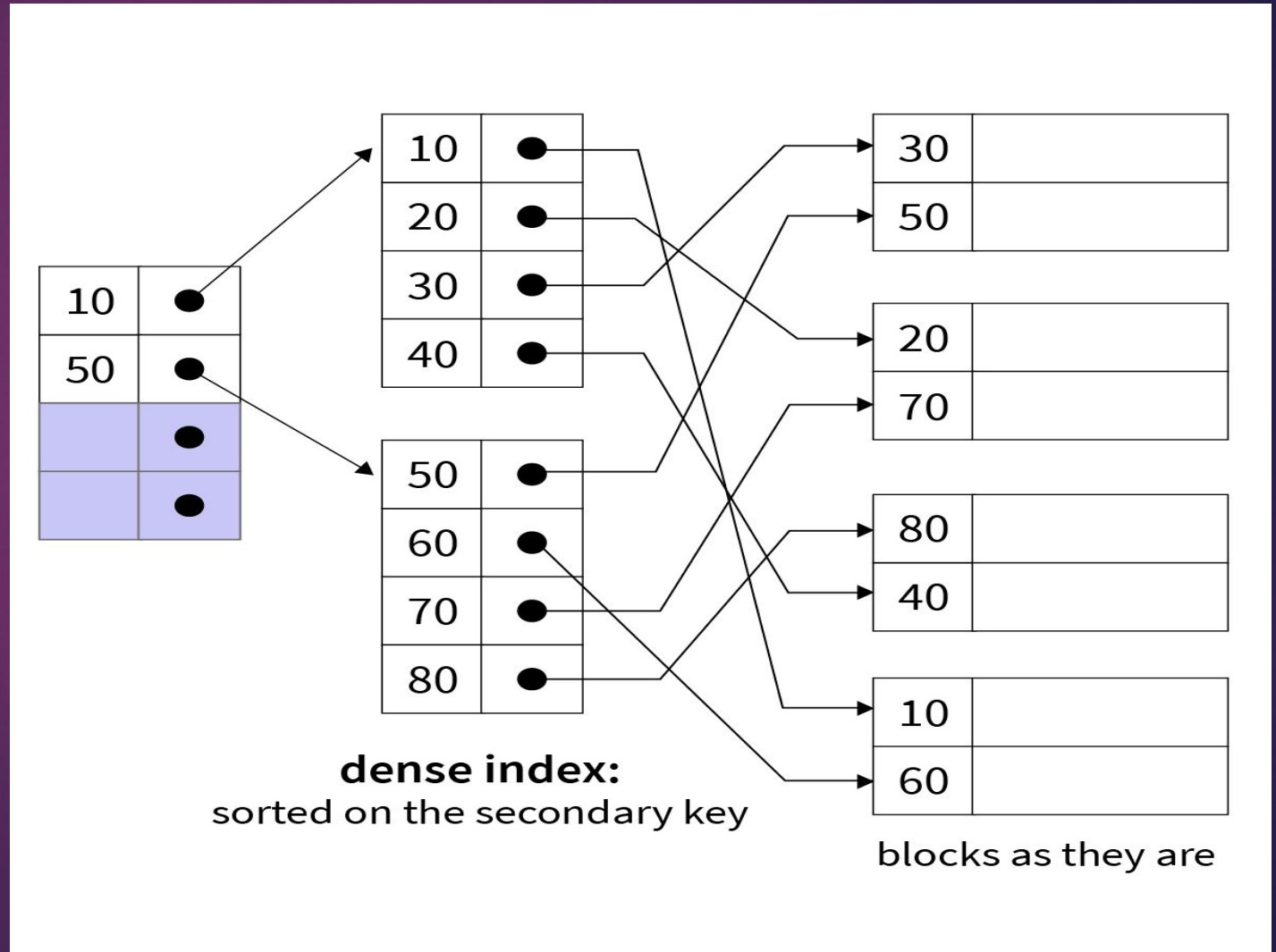
Example

- In case we want to find the record for roll 111 in the diagram, it will look for the highest item in the first level index that is equal to or smaller than 111. At this point, it will get a score of 100.
- Then it does $\max(111) \leq 111$ in the second index level and obtains 110. Using address 110, it now navigates to the data block and begins searching through each record until it finds 111.
- In this method, a search is carried out in this manner. In the same way, you can insert, update, or delete data.



Multi level/Secondary Indexing

- It is a two-level indexing technique used to reduce the mapping size of the primary index.
- The secondary index points to a certain location where the data is to be found but the actual data is not sorted like in the primary indexing.
- Secondary Indexing is also known as non-clustered Indexing.



Characteristics of Secondary Indexing:

- ▶ Search Keys are Candidate Keys.
- ▶ Search Keys are sorted but actual data may or may not be sorted.
- ▶ Requires more time than primary indexing.
- ▶ Search Keys cannot be null.
- ▶ Faster than clustered indexing but slower than primary indexing.

Lets see the syntax to store indexing in descending order-

- ▶ `CREATE INDEX index_name ON table-name (column-name_1, column-name_2 DESC);`
- ▶ **By default Sorting Order:**
- ▶ Character Data: Sorted by ASCII values of the characters.
- ▶ Numeric Data: Smallest to largest numbers.
- ▶ Date: Earliest date to the latest date.

Column and Functional:

- ▶ Generally, we prepare the index table with certain column values of the actual database but sometimes we can also use predefined SQL functions like UPPER() or LOWER() or MAX(), etc. to prepare the Search Keys.
- ▶ Example - We can convert all values in a column to uppercase and stored these results in the index.

- ▶ **Syntax:**

- ▶ `CREATE INDEX index-name`

`ON members(UPPER(target-column));`